

```
length: 1032 49153 > 5000)
Payload (size=1024)
```

As mentioned earlier, the trace event shown above corresponds to a packet enqueue operation because the line starts with the symbol '+'. The second line tells us the time at which the event happened. The third and fourth lines tell us about the node and device in which the event happened. We now know that the event happened in Device 0 of Node 1. Lines 5 and 6 tell us that a point-to-point protocol is used in this example. Lines 7 to 10 tell us about the IPv4 header details used in this example. We also get the IP addresses of the node from which the packet originates and the node to which it is destined. In this case, the packet originates from a node with IP address 192.169.100.1 and is destined to a node with IP address 192.168.100.1. Lines 11 and 12 tell us that about the UDP header. The thirteenth line tells us about the size of the payload. In this example, the size of the payload is 1024 bytes.

So now we have some idea about the data provided by the trace file *netanim1.tr*. But alas, the structure of a trace file is not always uniform. When a simulation involves different protocols and different applications, the resulting trace file also varies. So it is not possible to have a single formula to understand the different fields of an ns-3 trace file. Understanding the meaning of the different fields of an ns-3 trace file is one of the skills you have to acquire in the long journey to master ns-3. But whatever the structure of the trace file is, it will contain a number of fields representing some physical parameter. All you have to do is find out which field represents which physical parameter. But even then, you need some tools to analyse the trace file. The next section briefly discusses a few techniques to extract information from a trace file.

Tools to process trace files

Sometimes, there are millions of lines of data in the trace file. Then, manual processing becomes impossible. In such situations, we have different techniques to extract information from the trace file. Let us consider the simple problem of finding the number of UDP packets received at node 2, the receiver node. We will solve this problem using different approaches. One possible method is to use Linux commands. Highly useful Linux commands for text processing include *grep*, *cut*, *paste*, etc. The *grep* command is used to select lines from a text containing a particular pattern. The *cut* command is used to extract specific columns of data from a multi-column data file. The command *paste* is used for creating multi-column output files, especially useful for producing graphs. Now consider the one-liner given below.

```
grep "Ar" netanim1.tr | grep "NodeList/2" | wc -l
```

This will give you the number of packets received at

node 2. Here, we use pipelined Linux commands. First, we select all the lines starting with the letter 'r' — the event corresponding to packet reception — from the trace file. Then from this, we select the lines containing the pattern '/Node/2/'. And finally, we count the number of selected lines with the command 'wc -l'. But it may not always be possible to extract information using Linux commands alone. In many situations we need more powerful tools. One such tool is a programming language called AWK, optimised for text processing. Consider the AWK script *packet_count.awk* given below.

```
#!/bin/awk -f
BEGIN {
    count = 0;
}
{
    if($1 == "r" && $3 ~ /NodeList/2/)
    {
        count++;
    }
}
END {
    print "\n\nNo of packets received at the destination =
",count,"\n\n";
}
```

This AWK script, when executed, will give you the number of packets received at node 2. In an AWK script, the *BEGIN* section will be processed only once—in the beginning. The *END* section will also be executed just once—at the end. The code given between *BEGIN* and *END* sections, enclosed inside the curly bracket, will be executed on every line of the trace file being processed. Each line is divided into a number of fields with whitespace as the character deciding the ending of a field and the beginning of a new field. In the script, \$1 contains the first column and this column denotes the event type. \$3 contains the third column of the trace file and this corresponds to the third line of the trace file sample shown earlier. So the AWK script selects only those lines that denote a packet reception event at node 2. A counter is incremented if such an event happens and, finally, this value is displayed in the *END* section.

Even though the language AWK is highly suitable for text processing, its popularity has been waning over the years. As far as text processing is concerned, a general-purpose language called Perl has now taken centre stage. It will be highly beneficial in the long run if you select Perl rather than AWK for text processing. Given below is a Perl script *packet_count.pl* for finding the number of packets received at node 2.

```
#!/bin/perl
$infile=$ARGV[0];
```